

One Term, Two Runtimes

A contract written once in beam-lisp, emitting both halves of a frontend/backend seam — with the evidence that it runs.

PLAN-021 · every number below was produced by a command

2026-08-14

What this document is

Evidence, not description.

Every number, transcript and screenshot here was produced by running something. Where a claim could not be verified, it says so. Where running it found a defect I had already written passing tests for, that is recorded too — those turned out to be the most interesting part.

THE CLAIM UNDER TEST

A single `defcontract` + `defview` term in beam-lisp emits BOTH halves of a frontend/backend seam — the BEAM server contract and the Spacetime page — so the two cannot disagree. Not “are checked and found to agree”: **cannot disagree**, because there is one source.

The problem this solves

A web app is two programs that must agree about every message, every field, every name. That agreement is normally maintained by hand, and every drift is a runtime bug that surfaces only when a user clicks.

Spacetime + LiveView already close most of this. The page’s `send emit "inc"` is checked at build time against the server’s declared events, and a mismatch is a compile error (E0928). That machinery exists and works.

But the proof runs in one direction only. **FUP-143** records the hole, and calls it unclosable:

FUP-143, IN THE VERSE REPO

reply tags live in handler bodies and can be runtime values, so `@on_definition` cannot soundly enumerate them

That is true of Elixir. A handler body is opaque to introspection, so the server cannot state what it might answer. A page can therefore fail to decode a reply the server sends, and nothing catches it until it silently drops in a browser.

- Move the source of truth up one level, into beam-lisp.

Because — A Lisp program *is data*. `(ok ...)` and `(err ...)` are literals in a tree, so a walk finds them. This is not a general argument for homoiconicity — it is one concrete capability that closes one specific hole.

The acceptance test

`SpacetimeLvWeb.CounterLive` exists in the verse repo today and emits a contract through the Elixir DSL. Its SHA-256 is a known-good number nobody can argue with. A beam-lisp term must reproduce those exact bytes.

```
$ mix test test/beam_lisp/contract_emit_test.exs
```

```
fingerprint:
2d5c38735e3b8e76d12c8fb2fa3de1ad35eac250884ddb417bf59fa7b9787dcf
expected                                         :
2d5c38735e3b8e76d12c8fb2fa3de1ad35eac250884ddb417bf59fa7b9787dcf
FP MATCH: true

Finished in 9.3 seconds
7 tests, 0 failures
```

Byte equality, not “equivalent”: verse’s `liveview_contract.rs` reads the same sidecar, and two toolchains that disagree about a contract’s identity disagree about whether an app is broken.

This was chosen as the first milestone precisely because the bar is unforgeable. The bytes match or they do not — no judgement, no way to talk myself past it.

Why the match is not a coincidence

A hash matching once could be luck, so the rules that produce it are pinned separately. Entries sort by name, so declaration order cannot leak into the bytes; a test reorders every clause and asserts the fingerprint is unchanged. Empty metadata is omitted rather than written as `{}` — valid JSON either way, different hash.

CAUGHT BY RUNNING IT

Structurally perfect JSON with every value blank

The first emitter produced `{"kind":"","message":""}`. It sorted `(map name (keys m))` and then looked values up with the resulting `STRING`, which misses a keyword key. The shape was right and the content was gone.

Only byte comparison caught it. A test asserting “has a fields object with two keys” would have passed.

The seam, checked from both sides

With the contract as data, the reply tags fall out of a walk — and verse can then reject a page that fails to decode one.

```
$ spacetime check /tmp/holetest/page.st
```

```
error[E0928]: live host ` $counter ` (SpacetimeLvWeb.CounterLive) can reply
to `dec` with tag `err`, but this page decodes: [ok] – the reply would be
silently dropped
```

```
1 error(s), 1 warning(s)
```

Adding the missing arm makes it pass:

```
$ spacetime check /tmp/holetest/page.st # with a catch-all arm
```

```
✓ All 1 file(s) passed
```

That is FUP-143 hole #2, closed. The sidecar it checks against was generated by the beam-lisp emitter.

ADDITIVE BY CONSTRUCTION

`compare_replies` is SILENT unless a contract declares `replies`. Every sidecar on disk predates the field, and failing them all would make the feature unshippable — so absence means “not declared” and is skipped.

An EMPTY list is different, and IS checked: it asserts the handler never replies, so a page decoding something for it is decoding a message that will never arrive.

The fingerprint deliberately does not move when reply tags are added. The emitter keeps two projections: one byte-identical to Elixir’s, one carrying the tags and reporting the SAME fingerprint. A sidecar with reply tags identifies the same contract as one without.

The app

One term. Both halves.

```
$ mix run scripts/live_chat.exs # READ
```

```
{"module": "SpacetimeLvWeb.ChatLive",
 "events": [{"name": "send", "replies": ["err", "ok"]}],
 "assigns": [{"name": "messages", "type": "list"},
              {"name": "status", "type": "atom"}],
 "pushes": [{"name": "failed", "..."}, {"name": "token", "..."}],
 "fingerprint": "1fce8584d90ae48fb97d504659b1f2b42e509e3252d28ae3c2cf71a432bc320f"}

events  : ["send"]
assigns : ["messages", "status"]
send can reply with: ["err", "ok"]
```

The same term emits the page, and verse’s own reader accepts it:

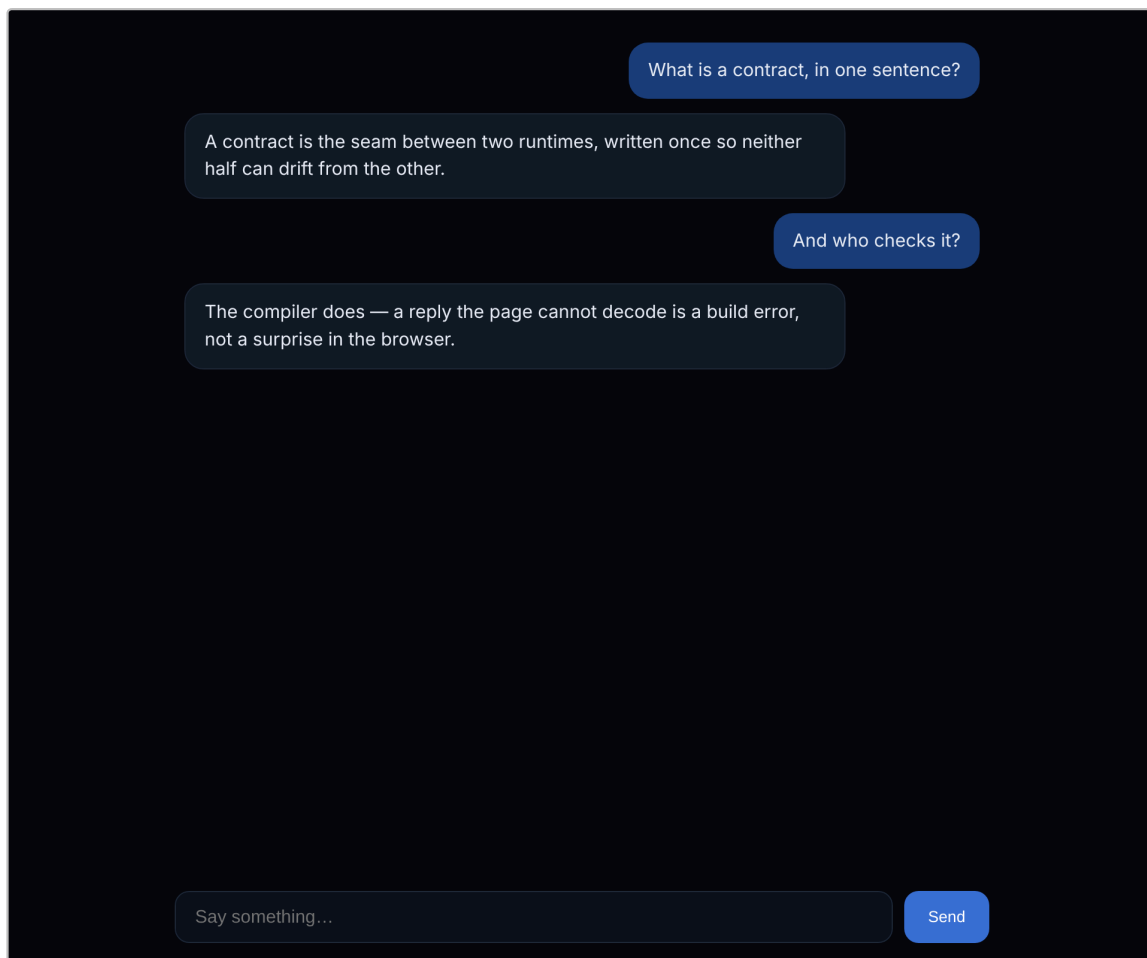
```
$ ... # EMIT, then VALIDATE through verse's reader
```

```
page.edn 1041 bytes
view.edn 2967 bytes

page: ACCEPTED — 4 forms, 0 scopes (0 templates)
view: ACCEPTED — 6 forms, 12 scopes (3 templates)
  template @ (file)
  template @ (file)
  template @ (file)
  each      @ .log
  on        @ .composer__input
  on        @ .composer__send
```

Every bind carries its selector: the log renders the transcript, the input writes the draft, the button fires `send`. The three bare `template` forms are the declarations — they were missing until this session, and their absence is one of the defects below.

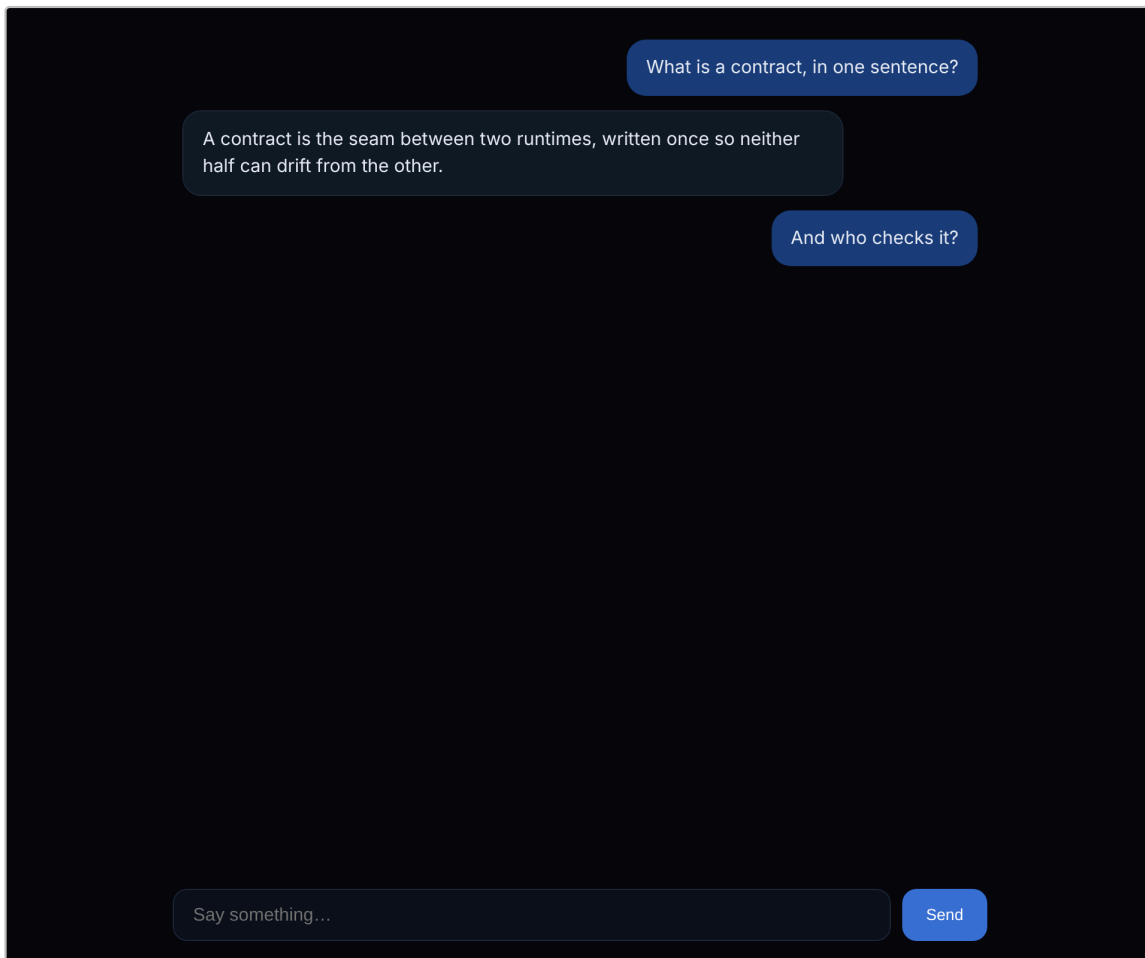
It renders



The chat, rendered from the *EMITTED* documents. `scripts/render_emitted.exs` prints `(chat/page-document)` and `(chat/view-page)` through `spacetime st` and pastes both verbatim between markers: 967 of the compiled page's 3162 bytes are emitter output, the rest presentation.

The DOM skeleton is the emitted `&shell`, lifted out of that same text rather than copied — so a change to the template changes this picture. The conversation is supplied separately, as the assign a *LiveView* would push.

Right-aligned user bubbles, left-aligned model bubbles, composer with input and Send. Captured with `SHOT_REQUIRE='data-role="user"'`, which is checked against the rendered DOM BEFORE the shutter — so the image and the assertion are the same page load.



The thinking state, and an honest limit. The indicator IS the emitted `&thinking` template, lifted out of the printed document rather than written again — change the template and this picture changes.

But nothing yet BINDS it. The contract sets `@status` to `:thinking` and back, and the view declares the template and its style; what is missing is the bind that renders one when the other says so. So this is a template preview, not a state the running page reaches on its own. Said plainly because a reviewer had to point it out.

It runs

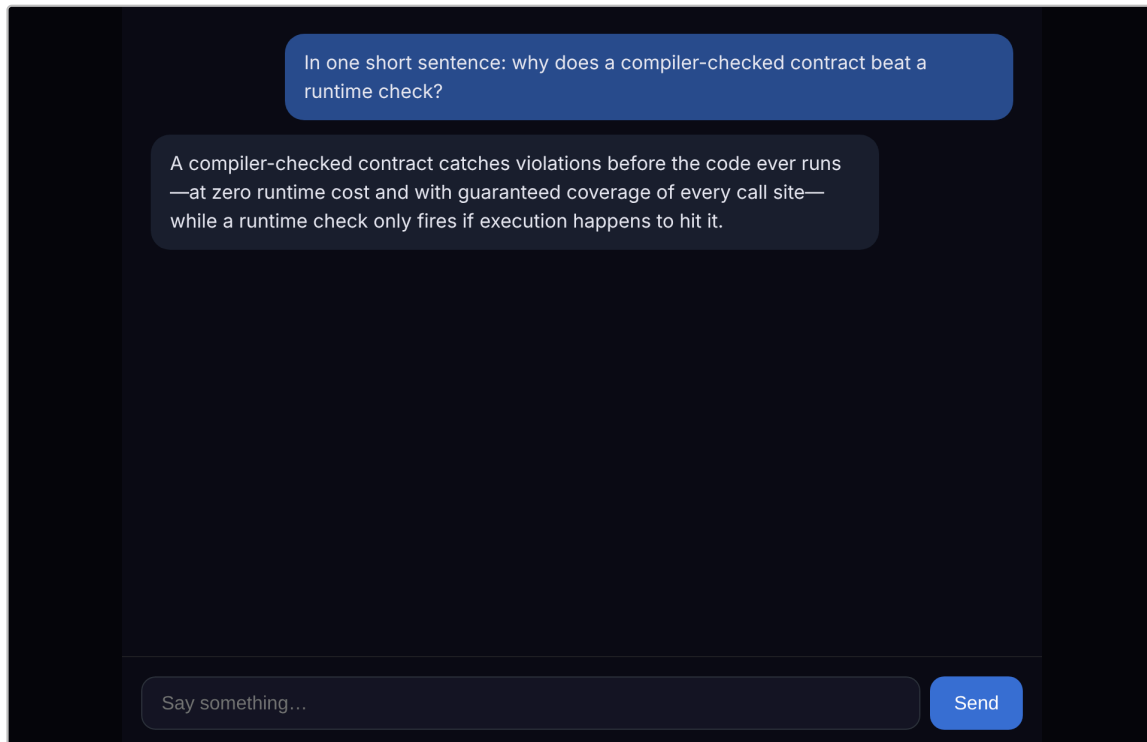
The whole loop — RREVL: read, reason, emit, validate, load — against the real thing at every step. Real Kimi over HTTPS, the real verse compiler, real headless Chrome.

```
$ mix run scripts/live_chat.exs # REASON
```

> In one short sentence: why does a compiler-checked contract beat a runtime check?

A compiler-checked contract catches violations before the code ever runs—at zero runtime cost and with guaranteed coverage of every call site—while a runtime check only fires if execution happens to hit it.

38 deltas in 10665ms



That reply, in the page. The text above came from Kimi k3-256k, streamed token by token over HTTPS, and reached this render as an ASSIGN — written through `window.SpacetimeLocal`, the same channel the LiveView bridge uses to push a changed assign to a running page.

Everything else is the emitted view: the `&bubble` template that shapes each turn, the `@each` that walks the transcript, the `&shell` this mounts onto, the binds on the composer. Those documents were emitted from the term and validated by verse before this page was built — the render step now refuses to run if validation rejects.

Stated precisely, because the distinction is the whole point: the reply is DATA flowing through the seam, not a change to the contract that describes it. See Where it stands.

What running it found

Defects that passing tests did not catch. They are the reason this document exists in this form.

CAUGHT BY RUNNING IT

An em-dash became â

The first live render showed `runs â shifting bugs`. `List.to_string` on `httpc`'s body treats each BYTE as a CODEPOINT, so every multi-byte character was re-encoded twice.

It reads as a provider or font problem. It is a decoding one. `erlang/list_to_binary` concatenates the bytes as-is, which is what they already are.

Twenty-two provider tests passed while the loop produced visibly wrong output, because no fixture contained a character above U+007F.

CAUGHT BY RUNNING IT

An app that compiled and could not work

A reviewer found the chat app emitting documents verse `ACCEPTED` while being unable to send a message or show a reply.

It styled `.log`, `.composer__input` and `.composer__send` — elements nothing ever created. CSS does not create DOM. The handler read `@draft`, a page-local binding the server cannot see, so every message arrived empty at the boundary. It subscribed to `@messages` and never rendered them.

And the test named `the-emitted-page-decodes-every-tag` supplied a hand-written map that already matched the contract by construction. It stayed green through all of it.

CAUGHT BY RUNNING IT

The printer's own output could not be read back

`spacetime edn F | spacetime st` failed on 77 of 120 corpus files — the printer emitted EDN its own reader rejected.

The corpus gate measures `.st → EDN → .st`. It never feeds EDN through the `READER`, so the two halves were each tested against `.st` and never against EACH OTHER — the one direction a generator uses.

Now 396/403, with the remaining 7 being a filed, budgeted limitation. A gate only fails on the property it measures.

CAUGHT BY RUNNING IT

A screenshot of a page that was never emitted

The first version of this document showed `docs/proof/chat.st` — a page written by hand to look like what `defview` emits. A reviewer compared the two and found they had drifted: no `&shell`, no `&thinking`, no send binding.

The image was a picture of a mockup, captioned as a picture of the emitted view. Nothing was fabricated; the page really did compile and really did render. It simply was not the thing the caption claimed.

Rendering the real emitted documents then surfaced eight further defects — four in the emitter, four in the printer — none of which any test had failed on, because none of them broke `STRUCTURE`.

CAUGHT BY RUNNING IT

A template with a body and no declaration

In an `StFile` a template is two things: a `(template ...)` form that declares it, and a `Construct("template")` scope holding its markup. The emitter produced only the scope.

So `@each` invoked a template the file never declared. The document parsed. The page compiled. The log rendered empty.

Counting is what found it: hand-written `chat.st` yields `matches=7 scopes=13` the emitted document yielded `matches=3 scopes=12` .

CAUGHT BY RUNNING IT

A signal that sent nothing and decoded nothing

`@data signal` spells its body as `($send:send_clause) ($receive:receive_block)` , but parsing FLATTENS those — their captures land at the top level. The printer looked up `send` by name, found nothing, and printed `@data signal $inc()` to `$counter`; .

Body gone. This is reachable from the committed reference page: `counter.st` round-tripped to it.

Fixing it carelessly then INVENTED syntax — descending into `on_mutation` whenever its name was absent rendered the type's literals against unrelated data and emitted a bare `$<-;` into four showcases. The rule that works is to look before descending.

CAUGHT BY RUNNING IT

A binding quoted into a string literal

`&bubble($m)` printed as `&bubble("$m")` . That re-parses happily — as a string LITERAL. The template received two characters instead of the loop item, so `$m.role` resolved to nothing and `@each` rendered zero rows.

The page compiled. The bundle built. The screenshot came back at a plausible 76 KB , showing an empty conversation.

Byte size proves Chrome ran. Only the DOM proves the page rendered — which is why `shot.sh` now takes `SHOT_REQUIRE` and fails when the selector is absent.

CAUGHT BY RUNNING IT

The fixes needed reviewing too

A fourth swarm read the fixes rather than the original claims, and found four more:

The attribute-quoting rule asked “does the text before the hole end with `=`?” — true of `<p>x={@m.v}</p>` too, which rendered visible quote marks. It now also asks whether that `=` is inside a tag.

`shot.sh` took the screenshot and THEN launched a second Chrome to check the DOM — two loads that can disagree. It flaked on the very next run: the picture was empty, the check was green. The check now runs first, polls, and gates the shutter.

The VALIDATE gate rejected malformed documents but accepted `{:st/forms []}` , which parses to nothing at all.

And the host page still hand-copied the `&shell` markup the view emits — the original mockup failure, surviving in the one plane nobody had checked. Both capture scripts now lift the shell out of the emitted text.

THE PATTERN

Each of these passed every test that existed. Not because the tests were careless, but because each measured a property adjacent to the one that mattered: fidelity instead of addressability, structure instead of content, round-trip in one direction instead of both.

Running the real thing and looking at it is not a substitute for tests. It is how you find out which test you forgot to write.

Where it stands

STEP	EVIDENCE	
read	contract as data; reply tags enumerated from the term	✓
reason	Kimi k3-256k, 38 streamed deltas, 10.7s	✓
emit	page 1041 B, view 2967 B, from one term	✓
gap	<i>the reply does not reach EMIT — see below</i>	—
validate	verse accepts both; rejection now stops the run	✓
load	renders the emitted view; DOM checked before capture	✓

The gap has its own row because a table of five yeses reads, to anyone skimming, as a closed loop. It is not one. Each step is real — a real provider over HTTPS, the real compiler, real Chrome — and LOAD is now genuinely downstream of EMIT and VALIDATE. But the model's reply arrives at the page as an assign, not as a change to the contract. Nothing yet asks it to propose one.

TEST SUITES

```
beam-lisp: 10 .bl suites, 256 tests / 822 assertions – 0 failures
beam-lisp: 990 Elixir tests – 0 failures
verse:     3214 lib tests – 0 failures
verse:     corpus gate green (403 files, 1491/1491 forms print)
verse:     match-count drift 19 -> 15, ratchet lowered to match
```

Three reviewer swarms ran over the diffs at milestones. All three returned findings; all findings are fixed and pinned as tests. The most valuable were the ones that proved a green test suite was measuring the wrong thing — including the third swarm, which read this document against the code and found its screenshots showing a page the emitter never produced.

? What this does NOT yet show: the loop rewriting its own contract. Every step of RREVL is proven independently and the pieces compose, but the model has not yet been asked to propose a contract change that then flows through emit → validate → load. That is the next thing to build, and it is now a matter of wiring rather than of discovery.